



INSTITUTO TECNOLÓGICO SUPERIOR DE LERDO

Nombre del alumno:

Alexander Adonai Molina Rodríguez - 242310131

Nombre del profesor:

Jesús Salas Marín

Nombre de la materia:

Administración y Organización de Datos

Carrera:

Ing. Informática

Fecha:

03/06/2026

Turno:

Matutino

EXPERIMENTACIÓN Y ANÁLISIS COMPARATIVO EN SISTEMAS DE ARCHIVOS (CSV VS. JSON)

1. Ecosistema del Proyecto y Competencia Semestral

Este proyecto final consolidó un entorno de pruebas experimentales de alta escala enfocado en una de las áreas más críticas de la gestión de información: los sistemas de salud pública y expedientes clínicos. El ecosistema desarrollado simula el flujo operativo de un complejo hospitalario masivo y está compuesto por un módulo de configuración global de variables aleatorias (`config.py`), un núcleo algorítmico central de benchmarking e interactividad (`main.py`) que genera dinámicamente un volumen de un millón de registros de pacientes, un set de librerías lógicas y operacionales (`funciones.py`, `reportes.py`), un módulo analítico-visual basado en `pandas` y `matplotlib` (`graficas.py`) que procesa métricas de distribución de edades, especialidades y niveles de riesgo, y un reporte comparativo en texto plano (`comparativa_resultados.txt`) derivado de pruebas de esfuerzo cronometradas en tiempo real.

La realización y el análisis de este experimento de ingeniería demuestran de forma directa que adquirí la competencia del semestre: "Evaluar diferentes organizaciones de archivos aplicándolas a situaciones reales". Al someter de forma simultánea los formatos estructurados tabulares (CSV) y jerárquicos estructurados (JSON) a una carga masiva de 1,000,000 de expedientes complejos, no solo programé código funcional, sino que evalué cuantitativamente el rendimiento de E/S (Entrada/Salida), el impacto de la serialización en disco y los costes computacionales asociados a la persistencia. Esto me otorgó el criterio técnico para diagnosticar los cuellos de botella de cada organización de archivos y dictaminar soluciones arquitectónicas basadas en las restricciones operativas del negocio.

2. Justificación de Formatos (Perspectiva de Rendimiento y Flexibilidad)

- **Organización Lineal Basada en Caracteres Separados por Comas (`pacientes.csv`):** Elegido debido a su óptima compactación y velocidad de transmisión para datos estructurados tabulares. Físicamente, al no contener metadatos repetitivos en cada registro y separar los atributos de los pacientes mediante comas simples, minimiza drásticamente el uso de almacenamiento secundario en disco. Científicamente, esto permite un volcado secuencial de alta velocidad y una integración perfecta con estructuras indexadas en memoria principal (`DataFrames`) mediante buffers optimizados en C, siendo ideal para la analítica masiva y el cálculo masivo de estadísticas de salud.
- **Organización de Objetos Notacionales Jerárquicos (`pacientes.json`):** Elegido debido a su alta flexibilidad estructural y riqueza semántica. A diferencia de las filas rígidas del CSV, JSON permite organizar los atributos del paciente de forma anidada, facilitando una representación fiel de un

expediente médico real (donde un paciente puede tener múltiples enfermedades o un histórico complejo de medicamentos en formato de lista). El costo en almacenamiento y ciclos de CPU para parsear texto y mapear diccionarios se justifica plenamente cuando el sistema requiere interoperabilidad, validación estricta de esquemas o una mutabilidad de datos ágil en entornos web.

3. Estimación de Uso y Escalabilidad (Análisis del Impacto de N Registros)

¿Qué le ocurre al sistema si el volumen de registros N escala a niveles industriales masivos?

- **Comportamiento en Operación de Escritura:** La fase de generación e inyección masiva revela dos tendencias claras. El archivo `pacientes.csv` demuestra un comportamiento lineal altamente eficiente debido a que la librería `csv.writer` realiza una escritura secuencial directa utilizando buffers intermedios, traduciéndose en tiempos mínimos de procesamiento en disco. Por el contrario, la serialización en `pacientes.json` incrementa drásticamente los costes computacionales. Debido a que el estándar JSON exige una estructura de arreglo global válida (`[...]`), la función `json.dump()` debe procesar y estructurar la totalidad del árbol de objetos en la memoria RAM antes de volcar el bloque de texto final al disco, lo que eleva el consumo de memoria de forma exponencial y expone al sistema a un desbordamiento de memoria principal si la RAM disponible es menor al tamaño final del archivo estructurado.
- **Comportamiento en Operación de Lectura y Consulta:** La fase de consulta de expedientes individuales a través de `pandas` expone una complejidad de orden lineal en ambos formatos para cargas en frío. Al ejecutar búsquedas por ID o nombre, el script debe volcar el bloque masivo de datos a memoria RAM. Si los datos escalan a decenas de millones de registros, el archivo CSV mantendrá una ventaja de tamaño en disco (ocupando aproximadamente un 40% menos de espacio físico que el JSON equivalente debido a la ausencia de claves repetidas por fila), agilizando los tiempos de transferencia de E/S. Sin embargo, una vez cargados en memoria dentro de un `DataFrame` indexado, la velocidad de consulta analítica se ecualiza gracias a la vectorización de operaciones, requiriendo en ambos casos la implementación futura de índices o fragmentación de archivos (`sharding`) para mitigar los tiempos de latencia del hardware de lectura.

4. Análisis Costo-Beneficio de la Arquitectura Híbrida

Criterio de Evaluación	Costo Técnico / Computacional	Beneficio Operativo
Persistencia Masiva (CSV)	Bajo: Pérdida total de jerarquía y flexibilidad estructural. Obliga a aplanar los datos anidados y no soporta de forma nativa tipos de datos complejos complejos por registro.	Muy Alto: Máxima velocidad de E/S y optimización extrema del espacio en disco. Es el rey indiscutible para almacenar registros históricos fríos y alimentar algoritmos de analítica.
Modelado de Expedientes (JSON)	Alto: Consumo masivo de memoria RAM durante la serialización y archivos físicos significativamente más pesados debido a la redundancia de las claves en cada fila.	Excelente: Representación fiel del mundo real. Permite que cada expediente clínico sea único, dinámico y auto-descriptivo, facilitando el intercambio de datos con APIs web hospitalarias.
Consultas e Interactividad (Pandas)	Moderado: Sobrecarga en los ciclos de la CPU para el parseo inicial de las cadenas de caracteres y la conversión de formatos de texto a tipos de datos nativos.	Muy Alto: Capacidad analítica robusta a nivel de microsegundos. Permite al personal médico calcular medias de temperatura, detectar riesgos de salud y generar reportes impresos al instante.

Conclusión

El desarrollo de este proyecto experimental y la culminación del portafolio de software representan un hito fundamental en nuestra formación como ingenieros informáticos. Antes de avanzar hacia el estudio formal de los Sistemas Gestores de Bases de Datos Relacionales (materia del siguiente semestre), esta investigación nos abrió los ojos sobre la infraestructura de bajo nivel que sostiene la persistencia de la información.

Comprendimos con rigor científico que una base de datos no es una entidad mágica ni abstracta, sino una capa de software sofisticada construida sobre los mismos principios físicos de manipulación de archivos planos y semiestructurados que

evaluamos en este curso. Aprendimos que en la ingeniería de software no existen herramientas perfectas ni "formatos mejores que otros" de forma absoluta; la excelencia en el diseño radica en la capacidad del ingeniero para evaluar las restricciones físicas del hardware, medir los tiempos de ejecución y seleccionar la organización de archivos adecuada para el modelo de negocio. Nos retiramos del semestre con la capacidad de estructurar soluciones de datos locales, robustas, eficientes y preparadas para escalar a arquitecturas empresariales complejas.